

Lab Report: Digital Image Processing & Computer Vision

MSc in Artificial Intelligence

Addis Ababa University

Prepared by: Zerihun Ayele Chala (GSR/0979/18)

Prepared for: Dr. Natnael Argaw Wondimu

Environment: Python 3 + OpenCV

Introduction

This report documents a complete set of image processing and computer vision laboratory exercises carried out using Python 3 and OpenCV on a custom dataset of 11 images. The dataset covers a diverse range of scene types: traffic lights, portraits, natural landscapes, road scenes, simple objects, peppers, fruits, flowers, parrots, low-contrast scenes, and night scenes. Each chapter covers a core area of the DIP/CV curriculum, progressing from basic image I/O through colour science, segmentation, morphology, and spatial filtering.

Chapter 1 — Image I/O and Colour Spaces

Exercise 1.1 — Load and Display All Images

All 11 images were loaded from disk using `cv2.imread()` and displayed in a 4-column grid. Each image was resized so its longest side did not exceed 600 pixels, preserving aspect ratio. Colour conversion from OpenCV's BGR order to RGB was applied before display with Matplotlib.

Exercise 1.1 — Our Image Dataset (11 images)

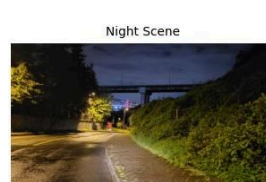
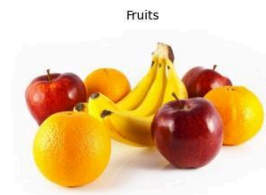
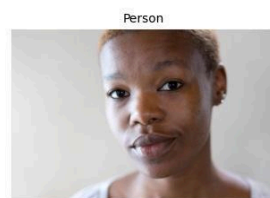


Figure 1.1 — The 11-image dataset displayed in a grid

Observation: The dataset is highly varied in content, colour distribution, and contrast level. Images such as 'Low Contrast' and 'Night Scene' will be more challenging for operations like thresholding and edge detection, while bright images like 'Fruits' and 'Parrot' are expected to respond well to most filters.

Exercise 1.2 — Save Images as PNG and BMP

Each image was saved in both PNG (lossless, compressed) and BMP (lossless, uncompressed) formats using `cv2.imwrite()`. Re-reading the saved files and checking their shapes confirmed that the pixel data was preserved identically across both formats. The key practical difference is file size: BMP files are significantly larger because they store raw pixel values without compression.

Observation: PNG is preferred for archiving processed outputs due to its lossless compression. BMP is useful when compatibility with legacy software is required or when avoiding any decompression overhead.

Exercise 1.3 — Colour Space Conversion: LAB and YCrCb

Each image was converted from BGR to both LAB and YCrCb colour spaces using `cv2.cvtColor()`. LAB separates lightness (L) from colour opponent channels (A: green-red, B: blue-yellow), making it perceptually uniform. YCrCb separates luma (Y) from chroma difference components (Cr, Cb), a format widely used in JPEG and video coding.

Exercise 1.3 — Color Spaces: Pepper



Figure 1.3 — Pepper image in original BGR, LAB, and YCrCb colour spaces

Observation: In the LAB image, the A and B channels encode colour hue information while L captures brightness independently — the pepper's red and green colours produce distinctly different A values. In YCrCb, Cr and Cb channels appear as pinkish-orange overlays because Matplotlib renders three-channel arrays as RGB; the raw channel values represent chrominance offsets, not displayable colour directly. These colour spaces are valuable for tasks such as skin detection (YCrCb) and perceptual colour distance measurement (LAB).

Exercise 1.4 — Channel Splitting with `cv2.split()`

`cv2.split()` was applied to both the BGR and HSV representations of each image to isolate individual channels. The R, G, and B channels were displayed with their respective colourmaps (Reds, Greens, Blues) and the H (Hue), S (Saturation), and V (Value) channels of HSV were displayed in greyscale.

Exercise 1.4 — cv2.split() Channels: Fruits

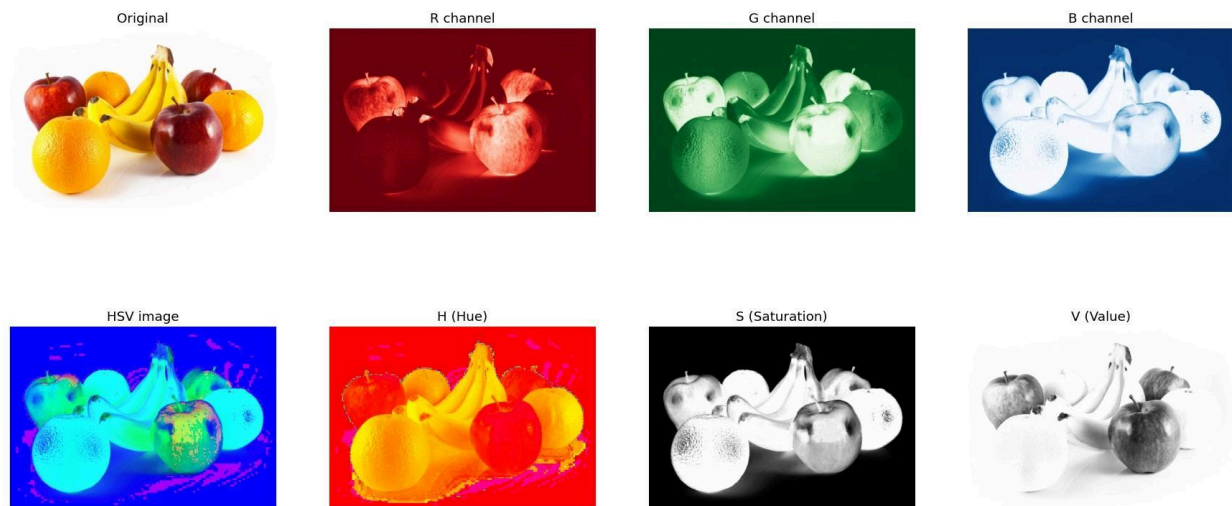


Figure 1.4 — BGR and HSV channel decomposition for the Fruits image

Observation: The Red channel shows high intensity for apples and oranges while the Blue channel is relatively weak across fruit surfaces. The Hue channel of HSV captures colour identity independently of lighting — oranges and bananas cluster at different hue values — making HSV particularly useful for colour-based segmentation. The Saturation channel is brightest where colours are most vivid and near-zero on the white background.

Chapter 2 — Colour Science and Point Operations

Exercise 2.1 — Grayscale Conversion: Luminosity Method

Three grayscale conversion methods were implemented and compared: (1) Luminosity method using ITU-R BT.709 weights ($0.2126 \cdot R + 0.7152 \cdot G + 0.0722 \cdot B$), (2) Simple average of the three channels (equal weights 0.333 each), and (3) OpenCV's built-in `cv2.cvtColor()` with `COLOR_BGR2GRAY` (which uses slightly different weights: $0.114 \cdot B + 0.587 \cdot G + 0.299 \cdot R$).

Exercise 2.1 — Grayscale Methods: Person

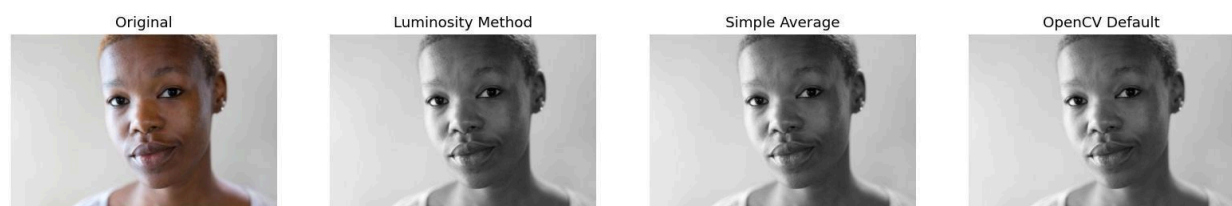


Figure 2.1 — Three grayscale methods compared on the Person image

Observation: The luminosity method produces the most perceptually natural result because it mirrors the human eye's sensitivity — we are most sensitive to green, moderately to red, and least to blue. The simple average overweights blue and underweights green, causing warm-toned skin to appear slightly lighter than expected. OpenCV's default is close to the

luminosity method but uses slightly different coefficients. The differences are subtle but visible in areas with strong red or blue hues.

Exercise 2.2 — Dynamic Brightness Adjustment

A brightness adjustment function was implemented that adds a constant delta value to every pixel and clips the result to $[0, 255]$. The function was demonstrated across delta values of $-100, -50, -25, 0, +25, +50,$ and $+100$, simulating a brightness slider.



Figure 2.2 — Brightness slider demonstration on the Traffic Light image

Observation: At $\text{delta} = -100$, the image becomes very dark and low-detail shadow regions are completely crushed to black. At $\text{delta} = +100$, highlight regions such as the lit traffic lights saturate and lose colour detail due to clipping. The middle range (± 25) provides useful contrast adjustment while preserving most tonal information. This simple additive approach is fast but does not adapt locally — a gamma curve or CLAHE would be more suitable for non-uniform scenes.

Exercise 2.3 — Per-Channel Histogram Equalisation

Histogram equalisation was applied independently to each of the three BGR channels using `cv2.equalizeHist()`, then the channels were merged back. This was compared to standard grayscale histogram equalisation. Both the resulting images and their histograms were plotted to visualise the redistribution of intensity values.

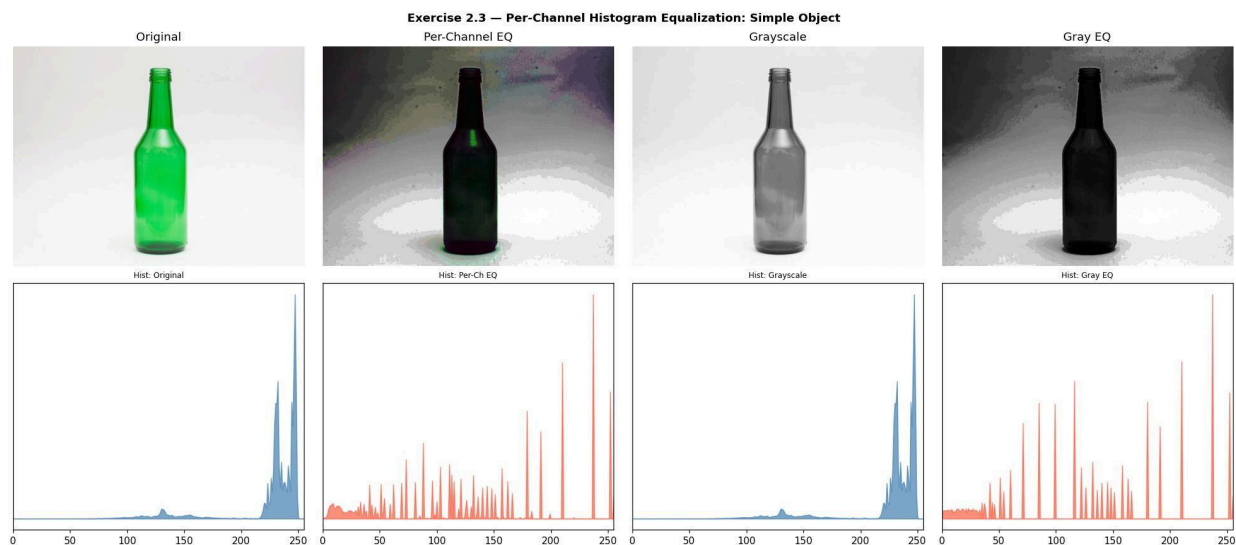


Figure 2.3 — Per-channel histogram equalisation on the Simple Object (green bottle)

Observation: The original histogram for the green bottle is heavily right-skewed (clustered at high values, i.e., the bright background). After per-channel equalisation, the histogram spreads across the full 0–255 range, as seen by the spike pattern typical of equalised images. Visually,

the background separates more sharply from the bottle but colour artefacts appear — the bottle shifts towards dark purple/blue tones — because equalising channels independently disrupts the balance between R, G, and B. Equalising in a luminance-separate space (e.g., LAB or YCrCb) would avoid this colour distortion.

Chapter 3 — Image Segmentation

Exercise 3.1 — Otsu's Thresholding

Otsu's method was applied to the grayscale version of each image using `cv2.threshold()` with the `THRESH_OTSU` flag. The algorithm automatically selects the threshold T that maximises between-class variance, effectively separating foreground from background without manual parameter tuning.

Exercise 3.1 — Otsu Thresholding: Fruits ($T = 176$)

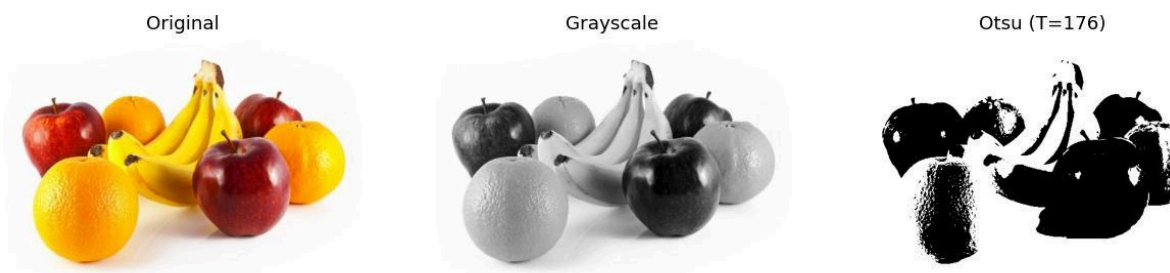


Figure 3.1 — Otsu thresholding on the Fruits image ($T = 176$)

Observation: For the Fruits image, Otsu selected $T = 176$, which cleanly separates the white background from the dark fruit surfaces. The binary result correctly identifies fruit blobs, though some fine details (stem, shadow edges) are lost. On images with poor bimodal histograms — such as the Night Scene and Low Contrast images — Otsu's method struggles because the intensity distribution is not cleanly bimodal, resulting in over- or under-segmentation. Otsu is most reliable when foreground and background pixel populations are comparable in size.

Exercise 3.2 — Edge Detector Comparison: Sobel, Prewitt, Canny

Three edge detectors were implemented and compared. Sobel uses a 3×3 gradient kernel with central-difference weighting (emphasises the centre row/column). Prewitt uses equal weights across all rows/columns. Canny is a multi-stage pipeline: Gaussian smoothing, gradient magnitude computation, non-maximum suppression (NMS), and hysteresis thresholding with `low=50` and `high=150`.

Exercise 3.2 — Edge Detector Comparison: Parrot



Figure 3.2 — Edge detector comparison on the Parrot image

Observation: Sobel and Prewitt produce similar results — broad, grey edge maps where thicker edges indicate stronger gradients. The Parrot's feather boundaries and eye are highlighted but there is significant noise in the feather texture region. Canny produces crisp, single-pixel-wide edges with far less noise because NMS thins edges to their strongest point and hysteresis discards weak, isolated responses. Canny is clearly the most precise detector for structured objects. The trade-off is that Canny requires two threshold parameters and is more sensitive to the smoothing sigma.

Exercise 3.3 — Region Growing Algorithm

A breadth-first-search (BFS) region growing algorithm was implemented from scratch. Starting from a manually specified seed pixel, the algorithm iteratively adds 4-connected neighbours whose absolute intensity difference from the seed value is within a threshold (thresh = 20). The grown region is returned as a binary mask and overlaid on the original image.

Exercise 3.3 — Region Growing: Low Contrast (seed=(133, 200), thresh=20)

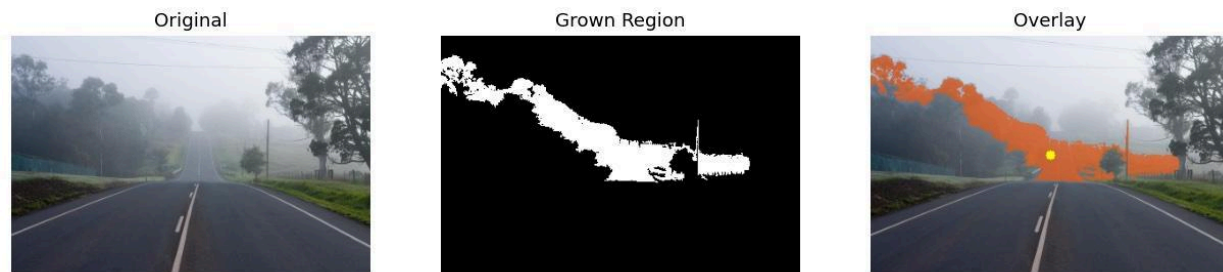


Figure 3.3 — Region growing on the Low Contrast image (seed=(133,200), thresh=20)

Observation: On the Low Contrast (foggy road) image, the region grows along the road surface as the pavement pixels share similar mid-grey intensity values. The overlay shows the grown region in orange, correctly capturing a connected portion of the road. Because the fog blends road and sky tones, the region leaks slightly into sky areas at the edges. Lowering the threshold would create a smaller, tighter region; raising it would cause excessive leaking into background regions. Region growing is highly seed-dependent — different seeds produce entirely different segmentations.

Chapter 4 — Morphological Image Processing

Exercise 4.1 — Structuring Element Shape Comparison

Four structuring element (SE) shapes of size 7×7 were compared: Square (MORPH_RECT), Disk/Ellipse (MORPH_ELLIPSE), Cross (MORPH_CROSS), and Diamond (custom kernel using L1-norm distance). Both erosion and dilation were applied to the binary version of each image, and results were displayed side by side.

Exercise 4.1 — SE Shape Comparison: Road



Figure 4.1 — SE shape comparison (Erode and Dilate) on the Road image

Observation: The Square SE is the most aggressive: erosion removes the most white foreground pixels and dilation expands the most. The Disk SE produces smoother, rounder boundaries because it matches circular structures in natural images. The Cross SE affects only horizontal and vertical directions, leaving diagonal corners less altered — visible as jagged edges on the truck contour. The Diamond SE is intermediate between the Cross and Disk. For binary morphology on natural images, the Disk SE generally gives the most visually natural results while the Cross is useful when only axis-aligned features should be affected.

Exercise 4.2 — Opening and Closing for Noise Removal

Salt-and-pepper noise at 8% density was added to Otsu-thresholded binary images. Three morphological operations using a 5×5 elliptical SE were then applied: Opening (erosion followed by dilation) removes salt noise (stray white pixels on a black background), Closing (dilation followed by erosion) fills pepper noise (dark holes within white regions), and combined Opening+Closing addresses both types simultaneously.

Exercise 4.2 — Noise Removal with Opening/Closing: Parrot

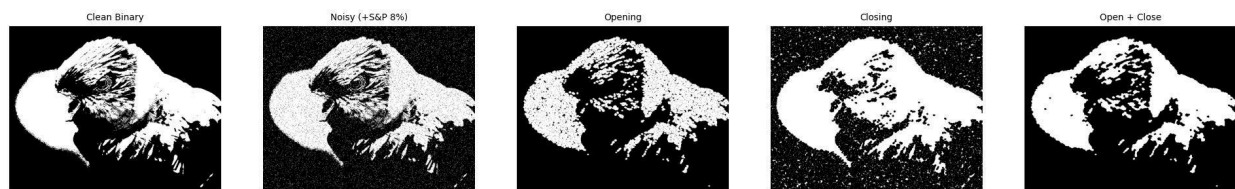


Figure 4.2 — Noise removal with opening and closing on the Parrot image

Observation: The noisy binary Parrot image contains scattered white specks (salt) on the black background and small black holes in the white foreground region. Opening effectively clears the scattered background specks. Closing fills the internal holes in the bird's body. The combined Open+Close result is the cleanest — most noise is removed and the bird silhouette is preserved. Some fine detail in the feather texture is inevitably lost due to the 5×5 SE size, illustrating the trade-off between noise suppression and detail preservation.

Exercise 4.3 — Morphological Feature Extraction

Top-hat (morphological white top-hat) and Black-hat transforms were applied using elongated structuring elements to extract fine structural features from images. A horizontal SE (25×1) was used to isolate horizontal bright lines (top-hat H), a vertical SE (1×25) for vertical features (top-hat V), a square SE (5×5) for the Black-hat (dark details on bright background), and a square SE for the Morphological Gradient (edge enhancement = dilation minus erosion).

Exercise 4.3 — Morphological Feature Extraction: Fruits

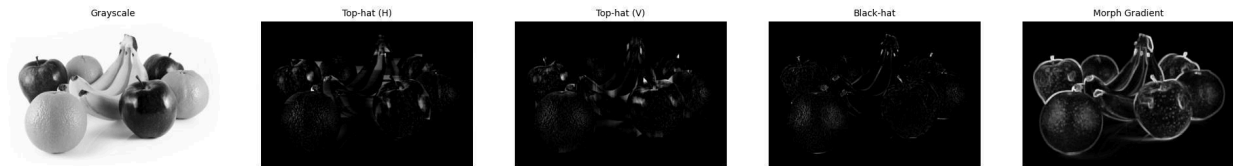


Figure 4.3 — Morphological feature extraction on the Fruits image

Observation: The top-hat transforms reveal fine bright details smaller than the SE — on the Fruits image, horizontal highlights on the fruit surfaces and the thin bright banana edges emerge against a black background. The Black-hat highlights dark features smaller than the SE (e.g., the apple stems, shadow details). The Morphological Gradient produces a clean edge map by measuring the local extent of the binary foreground — it is particularly responsive to the round edges of oranges and apples. These transforms are the foundation of document analysis pipelines where text strokes are extracted using elongated SEs.

Chapter 5 — Image Filtering and Correlation

Exercise 5.1 — Filter Size Study: 3×3 , 5×5 , 7×7

Box filters and Gaussian filters were applied at three kernel sizes (3×3 , 5×5 , 7×7) to the grayscale version of each image. For the Gaussian, sigma was set proportionally to kernel size (sigma = 1, 2, 3 respectively). Results were displayed in a 2×4 grid for visual comparison of blur strength.

Exercise 5.1 — Filter Size Study: Pepper

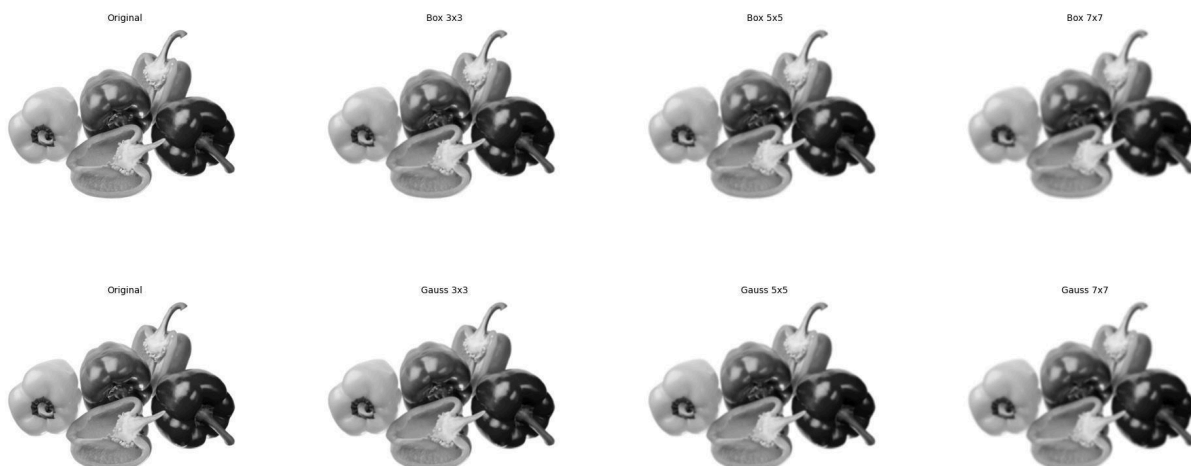


Figure 5.1 — Filter size study (Box and Gaussian) on the Pepper image

Observation: As filter size increases from 3×3 to 7×7 , both box and Gaussian filters produce progressively stronger blur. Fine details such as the pepper surface texture and stem are lost at 7×7 . The Gaussian filter preserves slightly more edge information than the box filter at the same size because it weights the centre pixel most heavily and transitions smoothly to the edge, whereas the box filter weights all pixels equally, including those far from the centre. This is visually apparent at 7×7 where box-filtered images look more uniformly smeared while Gaussian-filtered images retain a slight impression of structure.

Exercise 5.2 — Manual Unsharp Masking

Unsharp masking was implemented manually: a Gaussian blur ($\sigma=2$) was subtracted from the original to produce a detail mask, which was then added back scaled by a strength parameter. The formula is: $\text{sharpened} = \text{original} + \text{strength} \times (\text{original} - \text{blurred})$. Strength values of 0.5, 1.0, 1.5, 2.0, and 3.0 were tested.

Exercise 5.2 — Manual Unsharp Masking: Simple Object



Figure 5.2 — Manual unsharp masking on the Simple Object at increasing strengths

Observation: At strength = 0.5, the effect is subtle — edges become slightly crisper. At strength = 1.0 the bottle outline and label edges are noticeably sharper. At strength = 2.0 and 3.0, the sharpening becomes excessive and ringing artefacts (bright/dark halos) appear around high-contrast edges such as the bottle neck. This is the classic oversharpening artefact. In practice, strength values between 0.5 and 1.5 with an appropriate sigma provide the best perceptual sharpening without introducing visible artefacts.

Exercise 5.3 — Filtering Noisy Images: PSNR and SSIM Evaluation

Salt-and-pepper noise (5% density) was added to each grayscale image. Three filters were then applied: Box 3×3 , Gaussian 5×5 , and Median 5×5 . Image quality was quantified using PSNR (Peak Signal-to-Noise Ratio, in dB — higher is better) and SSIM (Structural Similarity Index, range $[0,1]$ — closer to 1 is better).

Exercise 5.3 — Filtering + Metrics: Simple Object



Figure 5.3 — Filtering noisy Simple Object with PSNR/SSIM scores

Observation: For the Simple Object, the noisy image achieves PSNR = 13.8 dB and SSIM = 0.104. The Box 3×3 filter improves this to PSNR = 22.1 / SSIM = 0.271 and the Gaussian 5×5 further to PSNR = 23.0 / SSIM = 0.335. However, the Median 5×5 filter dramatically outperforms both linear filters, reaching PSNR = 41.0 dB and SSIM = 0.991 — nearly perfect reconstruction. This is because salt-and-pepper noise produces isolated extreme outlier pixels; the median operation replaces each pixel with the middle value of its neighbourhood, inherently rejecting

outliers that linear averaging merely dilutes. The Median filter is the optimal choice for salt-and-pepper noise removal.

Exercise 5.4 — Template Matching via Patch Correlation

A 40×40 patch was extracted from the centre of each image and used as a template for normalised cross-correlation (`cv2.TM_CCORR_NORMED`) against the full image. The location of the maximum correlation coefficient was identified and a bounding box was drawn on the result image. The source patch location was also marked for comparison.

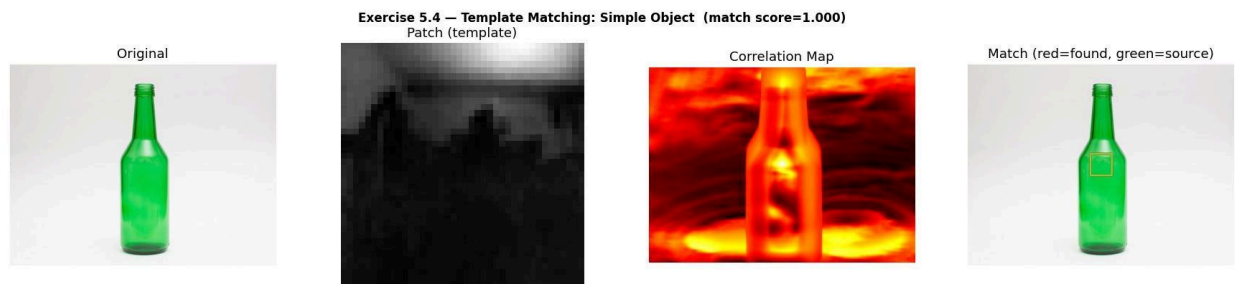


Figure 5.4 — Template matching on the Simple Object (match score = 1.000)

Observation: For the Simple Object image, the match score is 1.000 because the patch was extracted from the same image — the correlator finds an exact match at the source location. The correlation map (heat map) shows high values along the bottle body where similar grey tones exist, with the absolute peak at the true location. In a real application the template would come from a different image or a reference database. Normalised cross-correlation is robust to uniform lighting changes (additive offset) but not to scale or rotation changes, for which SIFT or ORB feature matching would be more appropriate.

Conclusion

This laboratory covered the full foundational pipeline of digital image processing: from loading and colour space transformation, through point operations and segmentation, to morphological analysis and spatial filtering. Several key insights emerged across the exercises.

Colour space choice is critical: working in LAB or YCrCb allows luminance and chrominance to be processed separately, avoiding colour distortion that occurs when operations like histogram equalisation are applied naively per BGR channel.

Segmentation performance is strongly image-dependent: Otsu's thresholding works well on high-contrast images with bimodal histograms but fails on low-contrast or multi-modal scenes. Canny edge detection consistently outperforms gradient-magnitude methods (Sobel, Prewitt) in edge precision due to its non-maximum suppression and hysteresis steps. Region growing provides flexible, connected-region segmentation but requires careful seed placement and threshold selection.

Morphological operations are powerful structural tools: the choice of structuring element shape and size directly determines which features are preserved or removed. For noise removal, combining opening and closing handles both salt and pepper artefacts effectively. Top-hat and morphological gradient transforms provide rich structural feature maps useful in document analysis and texture description.

For filtering, the Median filter is the clear winner for salt-and-pepper noise, achieving SSIM close to 1.0 while linear filters (Box, Gaussian) leave residual noise. Unsharp masking is an effective sharpening technique when used at moderate strength; excessive strength creates ringing artefacts. Template matching via normalised cross-correlation is a reliable method for locating known patches within images, though it is not invariant to scale or rotation.

Overall, these exercises demonstrate that no single algorithm is universally optimal — the right technique depends on the image characteristics, the type of noise or feature of interest, and the downstream task requirements.